

Configuración del servidor

```
const express = require("express");
const path = require("path");
const app = express();

// Configuración del puerto
const PORT = process.env.PORT || 3000;

// Configuración vistas y motor:
app.set("views", path.join(__dirname, "views"));
app.set("view engine", "ejs");

// Middleware estáticos y formularios
app.use(express.json());
app.use(express.urlencoded({extended: false})); // parseo de formularios
app.use(express.static(path.join(__dirname, "public")));

// Arrancamos servidor
app.listen(PORT, () => {
  console.log(`Servidor escuchando en el puerto: ${PORT}`);
})
```

package.json

```
{
  "scripts": {
    "start": "node ./bin/www",
  }
}
```

Explicación breve package-lock.json: Guarda el árbol exacto de dependencias y versiones instaladas, para que todo el equipo instale lo mismo y evitar "en mi PC funciona".

Socket.io

```
io.on('connection', (socket) => {
  console.log('Cliente conectado:', socket.id);

  socket.on('join-room', (roomName) => {
    // Unir al socket a la sala roomName
    socket.join(roomName);
  });

  socket.on('new-message', (msg) => {
    // Enviar msg a TODOS los clientes de la sala "chat-main"
    io.to('chat-main').emit('new-message', msg);
  });
});
```

Login y logout

```
//Renderizar la vidta login.ejs, lo haríamos en el documento app.js
```

```
app.get("/login", (req, res) =>{  
  return res.render("login", {title: "Login"});  
})
```

```
// Hacer el login posible
```

```
const database= require("../database");
```

ARRAY	OBJECT
<pre>app.post("/login", (req, res)=>{ const {username, password}= req.body; //el post envia siempre la informacion en el body mientras que el get en la url. //buscamos al usuario con ese nombre y contraseña const user= database.users.find(u=> u.username === username && u.password === password); //si encontramos al usuario entra y actualizamos la fecha de ultima conexcion if(user){ const lastLogin= new Date().toISOString(); req.session.user ={username : user.username, role : user.role}; return res.redirect("/"); } //si las credenciales son incorrectas return res.status(401).render("login", {title: "Login", error: "Credenciales incorrectas"}); })</pre>	<pre>app.post("/login", (req, res)=>{ const {username, password}= req.body; const uer= database.users[username]; if(user && password === user.password){ const lastLogin= new Date().toISOString(); req.session.user={username: user.username, role: user.role}; return res.redirect("/"); } return res.status(401).render("login", {title: "Login", error: "Credenciales incorrectas"}); })</pre>

```
//Hacer el logout
```

```
app.post("/logout", (req, res) =>{  
  req.session.user =null;  
  return res.redirect("/");  
})
```

Panel de administración

ARRAY	OBJECT
<pre> app.get("/admin", (req, res) => { const currentUser= req.session.user; // 1) Comprobar si hay usuario logueado y si es admin if (!currentUser currentUser.role !== "admin") { return res.redirect("/"); } // 2) Sacar la lista de usuarios de la "BD" const raw = database.users; // 3) Renderizar admin.ejs pasando la lista de usuarios return res.render("admin", { title: "Admin panel", users }); }); </pre>	<pre> app.get("/admin", (req, res) => { // 1) saco el usuario de la sesión const currentUser= req.session.user; // 2) si no hay user o no es admin, fuera if (!currentUser currentUser.role !== "admin") { return res.redirect("/"); } // 3) saco los usuarios de la BD const raw = database.users; // Si es objeto, lo convertimos a array const users = Array.isArray(raw) ? raw : Object.values(raw); // 3) Renderizar admin.ejs pasando la lista de usuarios, osea pinto la vista return res.render("admin", { title: "Admin panel", users }); }); </pre>
ARRAY	OBJETO
<pre> <table> <thead> <tr> <th> Username </th> <th> Role </th> <th> Last Login </th> <th> Action </th> </tr> </thead> <tbody> <% users.forEach(user => { %> <tr> <td> <%= user.username %> </td> <td> <%= user.role %> </td> <td> <%= user.lastLogin ? user.lastLogin : 'Never' %> </td> <td> <% if (user.role !== "admin") { %> <form action="/admin/deleteUser" method="POST" style="display:inline;"> <input type="hidden" name="username" value="<%= user.username %>"> <button type="submit">Delete</button> </form> <% } else { %> - <% } %> </td> </tr> </tbody> </table> </pre>	<pre> <table> <thead> <tr> <th> Username </th> <th> Role </th> <th> Last Login </th> <th> Action </th> </tr> </thead> <tbody> <% Object.value(users).forEach(user => { %> <tr> <td> <% user.username %> </td> <td> <% user.role %> </td> <td> <% user.latLogin ? user.lastLogin : 'Never' %> </td> <td> <% if (user.role !== "admin") { %> <form action="/admin/deleteUser" method="POST" style="display:inline;"> <input type="hidden" name="username" value="<%= user.username %>"> <button type="submit">Delete</button> </form> <% } else { %> - <% } %> </td> </tr> </tbody> </table> </pre>

<% }) %> </table>	<% }) %> </table>
----------------------	----------------------

- /login es la puerta de entrada: ahí sí miras username y password, comparas con la BD y, si todo está bien, guardas el usuario en algún sitio (por ejemplo req.session.user).
- /admin NO es otra puerta de login. Es una habitación privada a la que solo entra gente que ya pasó por la puerta y tiene la "pulserita de admin".
- Por eso, en /admin NO se vuelve a mirar username y password

Eliminar usuarios

ARRAY	OBJETO
<pre> app.post("/admin/deleteUser", (req, res) =>{ const {username} = req.body; const current= req.session.user; // 1) Comprobar que hay usuario logueado y que es admin if (!current current.role !== "admin") { // no tiene permiso return res.redirect("/"); } // En un array para borrar un elemento necesitamos su posicion //2) Buscamos en el array al usuario: const index= database.users.findIndex(u => u.username === username); if(index=== -1){ // usuario no existe -> volvemos al panel return res.redirect("/admin"); } const user= database.users[index]; //3) No permitir borrar admins if(user.role === "admin"){ return res.redirect("admin"); } return res.redirect("/"); }) </pre>	<pre> app.post("/admin/deleteUser", (req, res) =>{ const {username} = req.body; const current= req.session.user; // 1) Comprobar que hay usuario logueado y que es admin if(!current.role !== "admin" !current){ return res.redirect("/"); } // 2) Buscamos el usuario en el objeto const user = database.users[username]; if(!user){ //usuario no existe return res.redirect("/admin"); } 3) No permitir borrar admins if (user.role === "admin"){ return res.redirect("/admin"); } //4) Borramos el objeto delete database.users[username]; //5) Volver al panel return res.redirect("/"); }) </pre>

Menú común

Primero hacemos un middleware de global en app.js, para ver si todas las vistas tengan disponible el usuario actual:

```
// Middleware global: poner el usuario actual en res.locals
app.use((req, res, next) => {
  res.locals.currentUser = req.session ? req.session.user : null;
  next();
});
```

views/header.ejs:

```
<nav>
  <a href="/">Home</a>

  <% if (currentUser) { %>
    <span>Hola, <%= currentUser.username %></span>

    <% if (currentUser.role === "admin") { %>
      <a href="/admin">Panel admin</a>
    <% } %>

    <form action="/logout" method="POST" style="display:inline;">
      <button type="submit">Logout</button>
    </form>
  <% } else { %>
    <a href="/login">Login</a>
  <% } %>
</nav>
<hr>
```

Diferencia práctica entre app.locals y res.locals

- app.locals son **variables globales de la aplicación**: las defines una vez al arrancar el servidor y valen para **todas las peticiones y todas las vistas** (por ejemplo, el título de la web).
- res.locals son **variables por petición**: cambian en cada request. En el código tienes `res.locals.currentUser = req.session.user || null;`, así cada usuario ve su propio currentUser. En resumen: app.locals es "siempre igual para todos", res.locals es "depende de la petición / usuario".

Nombre global en vistas

```
app.locals.appTitle = "Nombre definitivo";
```

2. En donde aparezca el título en el html debemos cambiarlo por: `<title> <%= appTitle %> </title>`
3. Elimine el hardcode del nombre en las rutas.

```
app.get("/", (req, res) => {
  res.render("index", { title: "Inicio" });
});
```

```
app.get("/login", (req, res) => {
  res.render("login", { title: "Login" });
});
```

En lugar de app.locals, use un middleware que rellene res.locals en cada request

```
app.use((req, res, next) => {
  res.locals.appTitle = "Nombre definitivo";
  next();
});
```

Middlewares (checkAuth / checkAdmin) + ruta protegida

checkAuth

```
function checkAuth(req, res, next){
  if(req.session.user){
    next();
  }
  return res.redirect("/login");
}
```

Middleware checkLogin básico En routes/admin.js:

```
var express = require('express');
var router = express.Router();
var db = require('../database');

// Middleware checkLogin
function checkLogin(req, res, next) {
  if (!req.session.user) {
    req.session.error = 'Debes iniciar sesión para acceder al área de administración';
    return res.redirect('/login');
  }
  next();
}

// Aplicar el middleware a TODAS las rutas de /admin
router.use(checkLogin);

router.get('/', function(req, res) {
  res.render('admin_home');
});

router.get('/users', function(req, res) {
  const users = db.users.data;
  res.render('admin_users', { users: users });
});

module.exports = router;
```

checkAdmin

```
function checkAdmin (req, res, next){
  if(req.session.user.role === "admin"){
    next();
  }
  return res.status(403);
}
```

/admin incluyendo middleware

```
const database = require("../database");

app.get("/admin", checkAuth, checkAdmin, (req, res) => {
  const usersArray = Object.values(database.users);
  res.render("admin", { title: "Admin", users: usersArray });
});
```

Qué hace next() y qué pasa si no se llama?

- next() le dice a Express: **"este middleware ha terminado, pasa al siguiente middleware o a la ruta"**.
- Si **nunca** llamas a next() y tampoco haces res.send, res.render, res.redirect, etc., la petición se queda **colgada**: el navegador se queda cargando hasta que hace timeout, porque el servidor nunca envía respuesta.

Variaciones típicas Aplicar middleware a varias rutas: app.use("/admin", checkAuth, checkAdmin);

Hacer que currentUser esté en todas las vistas:

```
app.use((req, res, next) => {
  res.locals.currentUser = req.session.user || null;
  next();
});
```

API JSON

```
const database = require("../database");

app.get("/api/users", (req, res) => {
  try {
    const raw = database.users;

    //si la BD es un array lo dejamos asi si no lo pasamos a array:
    const arr = Array.isArray(raw) ? raw : Object.values(raw);

    const safe = arr.map(u => {
      const { password, ...rest } = u;
      return rest;
    });
  }
});
```

```

    res.json({ ok: true, data: safe });
  } catch (err) {
    res.status(500).json({ ok: false, error: "Error interno" });
  }
});

```

Hacer un fetch

```

const ul = document.getElementById("products");
const pError = document.getElementById("error");

async function loadProducts() {
  try {
    pError.textContent = "";
    ul.innerHTML = "";

    const res = await fetch("/api/products");

    if (!res.ok) {
      throw new Error("HTTP " + res.status);
    }

    const json = await res.json();

    if (!json.ok) {
      throw new Error(json.error || "Respuesta inválida");
    }

    json.data.forEach((p) => {
      const li = document.createElement("li");
      li.textContent = `${p.name} - ${p.price}€ (stock: ${p.stock})`;
      ul.appendChild(li);
    });
  } catch (err) {
    console.error(err);
    pError.textContent = "Error cargando productos";
  }
}

loadProducts();

```

Listado

```

const express= require ("express");
const app= express();
const database= require("./database")

app.get("/items", (req, res) => {
  try {
    const raw = database.items;
    const arr = Array.isArray(raw) ? raw : Object.values(raw);
  }
});

```

```

    return res.render("items", {
      title: "Listado",
      items: itemsArray
    });

  } catch (err) {
    res.status(500).json({ ok: false, error: "Error interno" });
  }
});

```

Búsqueda

```

app.get("/items", (req, res) => {
  try {
    const raw = database.items;
    let itemsArray = Array.isArray(raw) ? raw : Object.values(raw);

    const q = (req.query.q || "").trim().toLowerCase();

    if (q) {
      itemsArray = itemsArray.filter(item =>
        item.name.toLowerCase().includes(q)
      );
    }

    return res.render("items", {
      title: "Listado",
      items: itemsArray,
      q // opcional, por si quieres mostrar la búsqueda en la vista
    });
  } catch (err) {
    return res.status(500).send("Error interno");
  }
});

```

Listado

```

app.get("/products", (req, res) => {
  res.render("products", {title: "Productos", products: database.products});
});

```

Le pasas `products: database.products` para que en EJS puedas hacer un `forEach` y pintarlos.

Crear

```

app.get("/products/new", (req, res) =>{
  res.render("products-form", {
    title: "Nuevos productos",
    errors:[],
    valores:{name:"", price: "", stock: ""
  }
});
});

```

Esto solo muestra el formulario vacío. Pasas errors: [] (sin errores al principio) y values vacíos para rellenar inputs si lo necesitas.

```
app.post("/products", (req, res) => {
  const { name, price, stock } = req.body;

  const errors = [];
  const values = { name, price, stock };

  // Validaciones
  if (!name || !name.trim()) errors.push("El nombre es obligatorio.");

  const priceNum = Number(price);
  if (!Number.isFinite(priceNum) || priceNum <= 0) {
    errors.push("El precio debe ser un número mayor que 0.");
  }

  const stockNum = Number(stock);
  if (!Number.isInteger(stockNum) || stockNum < 0) {
    errors.push("El stock debe ser un entero mayor o igual que 0.");
  }

  if (errors.length > 0) {
    return res.status(400).render("product-form", {
      title: "Nuevo producto",
      errors,
      values
    });
  }

  // Crear producto (id incremental)
  const nextId =
    database.products.length > 0
      ? Math.max(...database.products.map(p => p.id)) + 1
      : 1;

  database.products.push({
    id: nextId,
    name: name.trim(),
    price: priceNum,
    stock: stockNum
  });

  return res.redirect("/products");
});
```

Editar

```

app.get("/products/:id/edit", (req, res) => {
  const id = Number(req.params.id);
  const product = database.products.find(p => p.id === id);

  if (!product) return res.sendStatus(404);

  return res.render("product-form", {
    title: "Editar producto",
    errors: [],
    values: {
      name: product.name,
      price: String(product.price),
      stock: String(product.stock)
    },
    productId: product.id // opcional para el action del form
  });
});

app.post("/products/:id", (req, res) => {
  const id = Number(req.params.id);
  const product = database.products.find(p => p.id === id);
  if (!product) return res.sendStatus(404);

  const { name, price, stock } = req.body;
  const errors = [];
  const values = { name, price, stock };

  if (!name || !name.trim()) errors.push("El nombre es obligatorio.");

  const priceNum = Number(price);
  if (!Number.isFinite(priceNum) || priceNum <= 0) {
    errors.push("El precio debe ser un número mayor que 0.");
  }

  const stockNum = Number(stock);
  if (!Number.isInteger(stockNum) || stockNum < 0) {
    errors.push("El stock debe ser un entero mayor o igual que 0.");
  }

  if (errors.length > 0) {
    return res.status(400).render("product-form", {
      title: "Editar producto",
      errors,
      values,
      productId: id
    });
  }

  product.name = name.trim();
  product.price = priceNum;
  product.stock = stockNum;

```

```

    return res.redirect("/products");
  });

app.post("/products/:id/delete", checkAuth, checkAdmin, (req, res) => {
  const id = Number(req.params.id);

  const idx = database.products.findIndex(p => p.id === id);
  if (idx === -1) return res.sendStatus(404);

  database.products.splice(idx, 1);
  return res.redirect("/products");
});

```

Eventos En el HTML:

```

<input id="txt" type="text">
<button id="btn"> Cargar </button>
<script rc="/app.js" defer> </script>

```

En app.js:

```

const btn = document.getElementById("btn");
const list = document.getElementById("list");
const txt = document.getElementById("txt");

// Listener 1
btn.addEventListener("click", () => {
  console.log("Click 1");
});

// Listener 2
btn.addEventListener("click", () => {
  console.log("Click 2");
});

// Añadir elemento a la lista
btn.addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = txt.value || "Elemento vacío";
  list.appendChild(li);
});

```

Validación en cliente En HTML:

```

<p id="msg"></p>

```

En app.js:

```
const msg = document.getElementById("msg");

btn.addEventListener("click", (e) => {
  // si está vacío, no añadimos y marcamos error
  if (!txt.value.trim()) {
    txt.classList.add("error");
    msg.textContent = "El campo no puede estar vacío";
    return;
  }

  // si está bien, quitamos error y mensaje
  txt.classList.remove("error");
  msg.textContent = "";
});
```

CSS:

```
.error {
  border: 2px solid red;
}
```